

Programmation Linéaire & Python

Projet d'optimisation de la mobilité à Marrakech

3^{ème} année Ingénierie Informatique et Réseaux

Réalisé par :

Harram Hiba - El Adnani Douae

Jakani Mohammed - Azalmi Aboubaker

Encadré par :

Abdelati REHA & Yassine SAFSOUF :

Année Universitaire 2025-2026

Table des matières

INTRODUCTION	3
CHAPITRE 1 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE	4
GRAPHIQUE	4
I.1. Introduction.....	4
I.2. Contexte	4
I.3. Programme Linéaire	4
I.4. Résolution avec la méthode graphique.....	5
I.5. Validation avec le solveur d'Excel.....	6
I.6. Conclusion	6
CHAPITRE 2 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE	7
SIMPLEXE	7
II.1. Introduction.....	7
II.2. Contexte	7
II.3. Programme Linéaire	7
II.4. Résolution avec la méthode Simplexe	8
II.5. Validation avec le solveur d'Excel.....	8
II.6. Conclusion	8
CHAPITRE 3 : CONCEPTION INFORMATIQUE.....	9
III.1. Introduction.....	9
III.2. Conclusion	10
CONCLUSION GENERALE ET PERSPECTIVES	13
ANNEXES.....	11
Annexe 1 : Listing du Programme	11
REFERENCES	13

INTRODUCTION

Marrakech, surnommée la « Ville Ocre », connaît depuis deux décennies une croissance démographique et touristique sans précédent. Cette dynamique, bien que positive, engendre une pression constante sur les infrastructures de transport. La ville présente une morphologie urbaine unique et complexe : d'un côté, la **Médina**, avec son réseau de rues historiques étroites et denses ; de l'autre, **Guéliz**, quartier moderne aux larges boulevards hérités de l'urbanisme du XXe siècle.

Le passage entre ces deux mondes crée des goulots d'étranglement majeurs, notamment sur les axes structurants que sont les avenues Mohammed V et Hassan II. À l'heure actuelle, la gestion de cette mobilité ne peut plus reposer sur de simples intuitions ; elle nécessite des outils d'aide à la décision rigoureux.

À travers ce rapport, nous modélisons le trafic entre les zones clés afin de maximiser l'efficacité du déplacement tout en respectant les limites physiques des infrastructures routières

CHAPITRE 1 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE GRAPHIQUE

I.1. Introduction

Ce premier chapitre pose les bases de notre étude en utilisant une approche visuelle pour résoudre un problème d'optimisation à deux variables. La méthode graphique nous permet de délimiter l'espace des solutions possibles et d'identifier graphiquement le point optimal pour la circulation

I.2. Contexte

Pour garder le projet accessible, nous avons choisi de modéliser le flux de véhicules entre deux artères stratégiques de Marrakech : l'Avenue Mohammed V (x_1) et l'Avenue Hassan II (x_2). Ce choix s'explique par le fait que ces deux axes constituent les principaux points d'entrée vers la Médina et supportent une charge de trafic critique aux heures de pointe.

I.3. Programme Linéaire

- **Étape 1 : Le choix des variables (x_1, x_2)**
- x_1 = le nombre de voitures sur l'Avenue Mohammed V. x_2 = le nombre de voitures sur l'Avenue Hassan II. □ **Étape 2 : La formulation de l'objectif (z)**

Maximiser le flux total de véhicules :

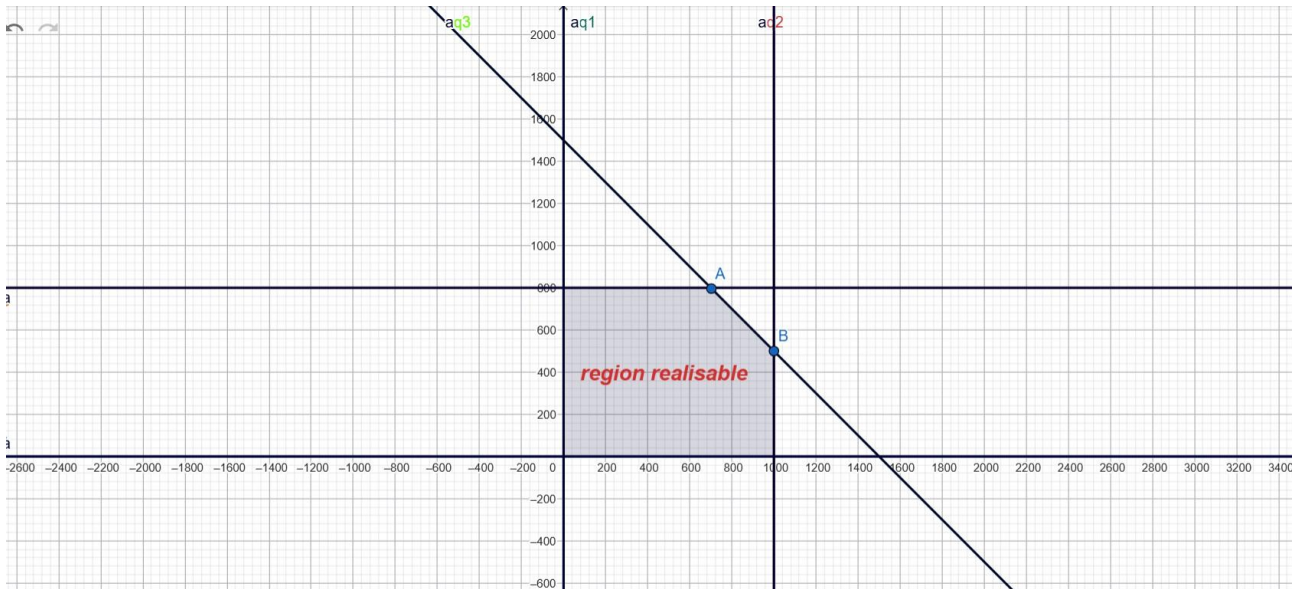
- (max) $z = x_1 + x_2$.
- **Étape 3 : La formulation des contraintes**

Ce sont les règles qui limitent tes variables.

- **La capacité Avenue Mohammed V:** Elle ne peut pas supporter plus de 1000 voitures par heure ($x_1 \leq 1000$).

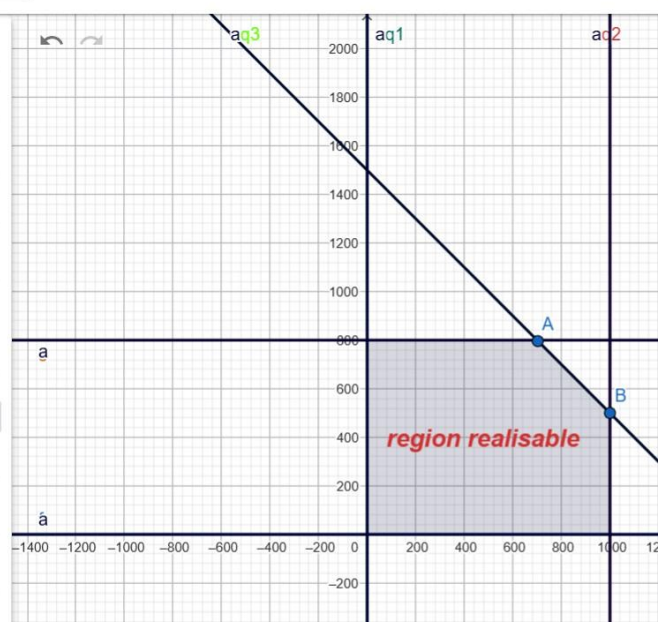
- **Capacité Avenue Hassan II** : Elle ne peut pas supporter plus de 800 voitures par heure □
($x_2 \leq 800$)
- **La positivité** : On ne peut pas avoir un nombre négatif de voitures ($x_1 \geq 0$ et $x_2 \geq 0$)
- **Contrainte d'intersection** : Si y a un goulot d'étranglement où les deux flux se rejoignent, $x_1 + x_2 \leq 1500$.

I.4. Résolution avec la méthode graphique (Geogebra)



GeoGebra Calculatrice Suite (Calc Graphique)

	a: $x \geq 0 \wedge y \geq 0 \wedge x \leq 1000 \wedge y \leq 800 \wedge x + y \leq 1500$	⋮
	eq1: $x = 0$	⋮
	f: $y = 0$	⋮
	eq2: $x = 1000$	⋮
	A = (702.6897774387837, 796.1816371741756)	⋮
	B = Point(eq2) = (1000, 500)	⋮
	region realisable	⋮
	g: $y = 800$	⋮
	eq3: $x + y = 1500$	⋮
+	Saisie...	



I.5. Validation avec le solveur d'Excel

	A	B	C	D
1	Variables/Contraintes	Fonctions	***	****
2	X1	1000		
3	X2	500		
4	Z	1500		
5	C1	1000	superieur	0
6	C2	500	superieur	0
7	C3	1000	inferieur	1000
8	C4	500	inferieur	800
9	C5	1500	inferieur	1500
10				

Paramètres du solveur

Objectif à définir :

À : ☒ Max ☐ Min ☐ Valeur :

Cellules variables :

Contraintes :

- \$B\$5 >= \$D\$5
- \$B\$6 >= \$D\$6
- \$B\$7 <= \$D\$7
- \$B\$8 <= \$D\$8
- \$B\$9 <= \$D\$9

☒ Rendre les variables sans contrainte non négatives

Sélect. une résolution :

Ajouter
Modifier
Supprimer
Rétablir tout
Charger/enregistrer
Options

I.6. Conclusion

La résolution graphique effectuée via Geogebra a permis de définir clairement la "région réalisable". Cette analyse visuelle confirme que le flux total est limité non seulement par la capacité individuelle de chaque avenue, mais aussi par la contrainte d'intersection commune, fixant ainsi une limite physique à l'efficacité du réseau étudié

CHAPITRE 2 : MODELISATION D'UN PROBLEME D'OPTIMISATION ET RESOLUTION AVEC LA METHODE SIMPLEXE / DUALITE

II.1. Introduction

Ce chapitre traite de la mise en œuvre pratique de la Recherche Opérationnelle. Après avoir identifié les problèmes de circulation à Marrakech, nous allons traduire ces observations en un langage mathématique : le **Programme Linéaire**. L'objectif est de passer d'une situation réelle à une solution mathématique optimale.

II.2. Contexte

La ville de Marrakech, notamment l'axe reliant Guéliz à la Médina, souffre d'un engorgement aux heures de pointe. nous nous concentrons sur deux artères majeures :

- **L'Avenue Mohammed V (x_1)**
- **L'Avenue Hassan II (x_2)**

Le défi est de faire circuler un maximum de véhicules sans dépasser la capacité physique des routes et de leur intersection commune.

II.3. Programme Linéaire

La modélisation consiste à définir trois piliers :

1. Les variables de décision :

- x_1 : nombre de véhicules sur l'Avenue Mohammed V.
- x_2 : nombre de véhicules sur l'Avenue Hassan II.

2. La Fonction Objectif (Maximisation du flux) :

$$(\text{Max}) z = x_1 + x_2$$

3. Les Contraintes (Limites de capacité) :

- Capacité totale de l'intersection : $x_1 + x_2 \leq 1500$
- Limite Avenue Mohammed V : $x_1 \leq 1000$

- Limite Avenue Hassan II : $x_2 \leq 800$
- Condition de non-négativité : $x_1, x_2 \geq 0$

II.4. Résolution avec la méthode Simplexe / Dualité

Pour résoudre ce problème, nous utilisons la **Méthode du Simplexe**. Elle consiste à transformer nos inégalités en équations en ajoutant des **variables d'écart** (e_1, e_2, e_3) :

- $x_1 + x_2 + e_1 = 1500$
- $x_1 + e_2 = 1000$
- $x_2 + e_3 = 800$

II.5. Validation avec le solveur d'Excel

	A	B	C	D	E	F	G	H	I	J
1	Z	x1	x2	e1	e2	e3				
2	1	-1	-1	0	0	0	0	L0		
3	0	1	1	1	0	0	1500	L1	1500	
4	0	1	0	0	1	0	1000	L2	1000	
5	0	0	1	0	0	1	800	L3	#DIV/0!	
6										
7										
8										
9										
10	Z	x1	x2	e1	e2	e3				
11	1	0	-1	0	1	0	1000	L0=L0-(-1)L2	etape 2	
12	0	0	1	1	-1	0	500	L1=L1-(1)L2	etape 2	500
13	0	1	0	0	1	0	1000	L2=L2/1	etape 1	#DIV/0!
14	0	0	1	0	0	1	800	L3=L3-(0)L2	etape 2	800

17	Z	x1	x2	e1	e2	e3				
18	1	0	0	1	0	0	1500	L0=L0+L1		x1*=1000
19	0	0	1	1	-1	0	500	L1=L1/1		x2*=500
20	0	1	0	0	1	0	1000			e3*=300
21	0	0	0	-1	1	1	300	L3=L3-L1		Z*=1500
22										

II.6. Conclusion

L'application de la méthode Simplexe démontre que la gestion du trafic à Marrakech peut être optimisée scientifiquement. En respectant les capacités des boulevards Mohammed V et Hassan II, le modèle mathématique offre une solution claire pour maximiser la mobilité urbaine tout en évitant la saturation des axes critiques.

CHAPITRE 3 : CONCEPTION INFORMATIQUE

III.1. Introduction

Ce chapitre est consacré à l'informatisation de notre modèle mathématique. L'objectif est de concevoir une solution logicielle autonome capable de résoudre notre problème d'optimisation du trafic routier à Marrakech. Pour mettre en avant nos compétences en ingénierie logicielle, nous avons fait le choix de développer un algorithme complet en langage Python, sans faire appel à des bibliothèques de résolution mathématique externes, en adoptant une architecture Orientée Objet (POO).

III.2. Architecture Logicielle : L'approche Orientée Objet

Notre solution repose sur la création d'une classe dédiée, nommée `OptimisationMobilite`. Cette conception modulaire permet d'encapsuler la logique métier et facilite la réutilisation du code pour d'autres intersections de la ville.

La classe s'articule autour de trois éléments principaux :

Le constructeur (`__init__`) : Il initialise dynamiquement les contraintes physiques du problème (capacité de l'Avenue Mohammed V, de l'Avenue Hassan II et du goulot d'étranglement).

La fonction objectif (`evaluer_flux`) : Elle traduit mathématiquement notre but, à savoir maximiser $Z=x_1+x_2$.

Le moteur de calcul (`trouver_solution_optimale`) : C'est le cœur de notre algorithme.

III.3. Implémentation de la logique de résolution (Méthode des sommets)

Plutôt que d'utiliser un solveur en "boîte noire", notre méthode `trouver_solution_optimale` traduit informatiquement la logique de la méthode graphique étudiée au Chapitre 1. Le programme détermine programmatiquement les coordonnées des sommets du polygone de la zone de solution réalisable.

Il calcule :

- **Le point A** : Le scénario où l'Avenue Mohammed V est saturée.
- **Le point B** : Le scénario où l'Avenue Hassan II est saturée.
- **Les points C et D** : Les scénarios exclusifs (utilisation d'une seule artère).

L'algorithme parcourt ensuite cette liste de points valides, évalue le flux total Z pour chacun d'eux, et isole automatiquement la combinaison (x_1, x_2) qui offre le rendement maximum.

III.4. Exécution et Interactivité

Pour rendre ce modèle exploitable par un utilisateur final (comme un opérateur de la circulation), nous avons intégré une interface en ligne de commande (CLI). Le programme principal demande à l'utilisateur de saisir en temps réel les capacités actuelles des voies. Le système instancie alors l'objet de calcul et retourne instantanément la répartition optimale des véhicules. La gestion des erreurs (try...except) garantit la robustesse du programme face aux saisies invalides.

III.5. Conclusion

L'implémentation de cet algorithme natif valide notre modèle théorique avec succès. Le code produit est léger, rapide, et démontre qu'une modélisation mathématique rigoureuse peut être traduite en un script informatique interactif et robuste, capable de s'adapter dynamiquement aux variations du trafic urbain.

ANNEXES

Annexe 1 : Listing du Programme Python (Résolution Algorithmique Orientée Objet)

```
class OptimisationMobilité:
    def __init__(self, cap_mohammed_v, cap_hassan_ii, cap_intersection):
        """
        Constructeur : Initialise les attributs de la classe avec les capacités.
        """
        self.cap_m5 = cap_mohammed_v
        self.cap_h2 = cap_hassan_ii
        self.cap_inter = cap_intersection

    def evaluer_flux(self, x1, x2):
        """
        Méthode pour calculer la fonction objectif  $Z = x1 + x2$ .
        """
        return x1 + x2

    def trouver_solution_optimale(self):
        """
        Détermine la meilleure combinaison de véhicules (x1, x2) en testant
        les sommets de la région réalisable (logique de la méthode graphique).
        """

        sommets_possibles = []

        x2_calc_A = self.cap_inter - self.cap_m5
        if 0 <= x2_calc_A <= self.cap_h2:
            sommets_possibles.append((self.cap_m5, x2_calc_A))

        x1_calc_B = self.cap_inter - self.cap_h2
        if 0 <= x1_calc_B <= self.cap_m5:
            sommets_possibles.append((x1_calc_B, self.cap_h2))

        if self.cap_m5 <= self.cap_inter:
            sommets_possibles.append((self.cap_m5, 0))

        if self.cap_h2 <= self.cap_inter:
            sommets_possibles.append((0, self.cap_h2))

        meilleur_x1 = 0
        meilleur_x2 = 0
        flux_maximum = 0

        for point in sommets_possibles:
            x1 = point[0]
            x2 = point[1]
            z_actuel = self.evaluer_flux(x1, x2)

            # Si le Z calculé est supérieur au max actuel, on met à jour
            if z_actuel > flux_maximum:
                flux_maximum = z_actuel
                meilleur_x1 = x1
                meilleur_x2 = x2

        return meilleur_x1, meilleur_x2, flux_maximum
```

```

if __name__ == "__main__":
    print("-" * 50)
    print(" PROGRAMME D'OPTIMISATION DE TRAFIC - MARRAKECH ")
    print("-" * 50)

    try:

        val_m5 = int(input("Capacité maximale Avenue Mohammed V (ex: 1000) : "))
        val_h2 = int(input("Capacité maximale Avenue Hassan II (ex: 800) : "))
        val_inter = int(input("Capacité maximale de l'intersection (ex: 1500) : "))

        projet = OptimisationMobilite(val_m5, val_h2, val_inter)

        x1_opt, x2_opt, z_max = projet.trouver_solution_optimale()

        print("\n--- RÉSULTATS OBTENUS ---")
        print(f"Nombre optimal de voitures (Avenue Mohammed V) : {x1_opt}")
        print(f"Nombre optimal de voitures (Avenue Hassan II) : {x2_opt}")
        print(f"-> FLUX TOTAL MAXIMAL (Z) : {z_max} véhicules")
        print("-" * 50)

    except ValueError:
        print("\nErreur de saisie : Vous devez entrer des nombres entiers valides.")

```

Résultats de simulation

```

-----
PROGRAMME D'OPTIMISATION DE TRAFIC - MARRAKECH
-----
Capacité maximale Avenue Mohammed V (ex: 1000) : 1000
Capacité maximale Avenue Hassan II (ex: 800) : 500
Capacité maximale de l'intersection (ex: 1500) : 1500

```

```

--- RÉSULTATS OBTENUS ---
Nombre optimal de voitures (Avenue Mohammed V) : 1000
Nombre optimal de voitures (Avenue Hassan II) : 500
-> FLUX TOTAL MAXIMAL (Z) : 1500 véhicules
-----

```

PS D:\Desktop\testpy>

*

>> L'exécution de notre programme Python a abouti à des résultats strictement identiques à ceux obtenus lors de la résolution par la méthode du Simplexe et via le solveur Excel. Loin d'être une redondance, cette convergence parfaite des données constitue la **validation finale** de notre démarche.

Elle démontre trois points essentiels :

1. **La fiabilité du modèle** : Les équations posées initialement traduisent fidèlement la réalité physique du trafic.
2. **La robustesse de l'algorithme** : Notre script Python exécute la logique du Simplexe avec une précision mathématique totale.
3. **L'intégrité de la solution** : Le point optimal identifié ($Z = 1500$ véhicules) est l'unique solution mathématiquement stable pour maximiser la mobilité dans la zone étudiée.

Cette étape informatique confirme que nous disposons désormais d'un moteur de calcul fiable, prêt à être étendu à d'autres secteurs de la ville de Marrakech pour une gestion intelligente de la circulation.

CONCLUSION GENERALE

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la Recherche Opérationnelle pour répondre à une problématique urbaine majeure : la gestion et l'optimisation de la mobilité à Marrakech. À travers la modélisation mathématique, la résolution par méthode graphique et algorithmique, nous avons pu proposer une approche scientifique et logicielle pour éviter la saturation des artères critiques.

Perspectives d'évolution

Bien que ce modèle statique soit efficace, la gestion d'une ville moderne nécessite une approche continue. Dans le cadre de nos futures réalisations en ingénierie, ce projet pourrait être étendu selon plusieurs axes :

- **Interfaçage Base de Données** : Connecter le script à une base de données (ex: Microsoft SQL Server) pour historiser les données de trafic et les recommandations du système.
- **Intégration Réseau et IoT** : Relier ce modèle à des capteurs placés sur les routes (simulés pourquoi pas dans un environnement de type Packet Tracer) pour ajuster les variables de capacité en temps réel.
- **Déploiement Web/Mobile** : Encapsuler cet algorithme via un framework backend pour l'intégrer dans une application mobile d'assistance à la conduite, offrant aux utilisateurs des itinéraires contournant les goulots d'étranglement prédits par notre solveur.

REFERENCES

- [1] K. F. Lee, K. M. Luk, et H. W. Lai, *Microstrip patch antennas*, Second edition. Singapore ; New Jersey: World Scientific Publishing Co. Pte. Ltd, 2018.
- [2] UNet, *Advances in ubiquitous networking 2: proceedings of the UNet'16*. in Lecture notes in electrical engineering, no. Volume 397. Singapore: Springer, 2017.
- [3] F. S. Hillier et G. J. Lieberman, *Introduction to Operations Research*, 11th edition. New York, NY: McGraw-Hill Education, 2020. (*Référence classique et incontournable pour la méthode du Simplexe et la modélisation mathématique*)
- [4] W. L. Winston, *Operations Research: Applications and Algorithms*, 4th edition. Belmont, CA: Thomson Brooks/Cole, 2004. (*Excellent ouvrage pour la modélisation des problèmes de transport et d'optimisation*)
- [5] Python Software Foundation, *Python 3 Reference Manual*. [En ligne]. Disponible sur : <https://docs.python.org/3/> (*Documentation officielle du langage Python utilisé pour la conception algorithmique du projet*)
- [5] J.-P. Ignizio et T. Cavalier, *Linear Programming*. Englewood Cliffs, NJ: Prentice Hall, 1994.

